

Production lines — Function design

005_DD-FN implements backend processing for FN-032–FN-036 and REQ-064–REQ-069. WI-009 approved plan revision 1, step 8. Design only; no application method, SQL or migration has been implemented/executed.

1. Document control and references

Field	Value
Document ID / version	005_DD-FN / 1
System / subsystem	ProductionManagementAI / Master data
Module	ProductionLineService and order eligibility integration
Work item / author	WI-009 / Agent
Created / updated	2026-10-01 / 2026-10-01
State	Submitted for review; not implemented

Version	Date	Author	Change
1	2026-10-01	Agent	Application methods, locking, atomicity, commit outcomes and instrumentation

Inputs: approved [005 BD](#), [005 DB](#), [main DD](#) and [API version 2. Brief](#), [decisions](#) and [plan](#) remain authoritative. This file owns backend sequences; API field catalogs stay in API, UI states in main DD and later SPD. The new order-impact DD remains a later review step.

2. Method index and shared dependencies

All method names below are proposed, created/updated by Agent on 2026-10-01. Controller authorization uses confirmed Admin/Operator ProductionOrderEditor for all seven feature endpoints. Methods are reached through authorized controllers; no anonymous job or new external service bypass is introduced.

Function / API	Method	Return / responsibility
FN-032 / API-PL-01	ProductionLineService.ListAsync	<code>Result<LinePage></code> ; bounded snapshot list
FN-033/034 / API-PL-02	GetAsync	<code>Result<LineDetail></code> ; parent and paged pairs

Function / API	Method	Return / responsibility
FN-033/034 / API-PL-03	CreateAsync	Result<LineDetail> ; atomic new aggregate
FN-033/034 / API-PL-04	UpdateAsync	Result<LineDetail> ; versioned aggregate edit
FN-035 / API-PL-05	RetireAsync	Result<LineSummary> ; one-way retirement
FN-034 / API-PL-06	ProductChoicesAsync	Result<ProductChoicePage> ; active unassociated products
FN-036 / API-PL-07	EligibleAsync	Result<EligibleLinePage> ; current eligibility hints
FN-036 / existing orders	ValidateOrderLineAsync	Result<OrderLineDecision> ; presence/history/start rules in existing write transaction

Shared dependency	Responsibility / boundary
IProductionLineRepository	EF reads, parent/pair tracking, parameterized locks and save; same scoped ApplicationDbContext for a write
Existing order repository/transaction	Order xmin, quantity/status/field locks, number issuer and atomic order persistence retained
Product observation port	Current unit/revision/state read under product lock; no HTTP Product master calls
TimeProvider / plant clock	UTC master audit stamps; existing order plant-time rules remain
Validation/Result/problem mapper	Typed errors and API mappings; no exception/SQL text returned
ProductionLineTelemetry	Single classified completion and API-approved source/meter

Request data to external APIs: none. Repository/transaction ports are in-process, not new integrations. Reuse EF/Npgsql and existing parameterized raw-lock pattern; no ORM/dependency, architectural layer or generic retry framework added.

Shared input and return mapping

Data	Internal handling / output mapping
Code/name/query	Trim, count Unicode code points; query wildcard escaping; DB collation governs comparisons
Hours/minutes strings	API syntax/range validated again in Application; invariant decimal, no floating-point conversion or rounding
Parent version	Parse canonical uint string; compare locked xmin; response invariant opaque string
Unit revision	Parse canonical bigint string; observed value comparison; confirmed revision set from locked product only
Active state / unit warning	Stored states; requiresUnitConfirmation = unit mismatch OR revision mismatch
UTC timestamps / decimals	Z timestamp; API canonical decimal string formatter, no gratuitous trailing zeros

Data	Internal handling / output mapping
Order lineId	Presence-aware value distinguishes omitted from explicit null; existing order version/quantity formats unchanged

3. Transaction and lock primitives

3.1 Read snapshots and deadlines

List/detail/choices/eligible count and rows execute in one read-only REPEATABLE READ transaction per result. Set read-only before data queries. Projected joins are deliberate: pair/product current-unit and order/line names need one coherent snapshot without N+1 reads. AsNoTracking for projections; no snapshot persists across browser pages. Integrity failure is not empty data.

All feature methods link request cancellation with the approved 15-second use-case deadline. Transaction-local statement_timeout=10s and lock_timeout=5s are set before data queries; provider command waits must also respect the remaining use-case deadline. These are planned settings, not configured runtime evidence. Do not change global defaults or existing endpoint deadlines. A transaction cleanup attempt uses a fresh bounded token (proposed 2 seconds), since the original request token may already be cancelled; this is cleanup, not a second operation. A client deadline or cancellation never establishes whether a write committed.

Read behavior and settings follow PostgreSQL documentation: [isolation](#) and [timeouts](#).

3.2 Write lock order

READ COMMITTED writes acquire products by ascending canonical UUID, then line, then saved pairs by ascending product UUID. Canonical lower-case D-format UUID ordinal sorting matches byte-order intent; every new writer uses the same order. Only touched products/pairs need locks; untouched associations remain unchanged. Do not acquire product locks after parent locks or upgrade shared line locks.

SQL below is design notation; ;parameters denote driver-bound values. It is not an executable migration or tested query. EF lock materialization uses the existing FromSqlInterpolated pattern; materialize before further composition and avoid an already tracked stale instance replacing the newly locked observation.

```

-- For each touched product, in sorted order; observe after any lock wait.
SELECT p.*, p.xmin FROM products p
WHERE p.id = :product_id FOR SHARE;
-- Aggregate edit/retirement: track locked parent and its fresh xmin.
SELECT l.*, l.xmin FROM production_lines l
WHERE l.id = :line_id FOR UPDATE;
-- Existing pair action, after parent lock; no independent editor token.
SELECT lp.* FROM production_line_products lp
WHERE lp.line_id = :line_id AND lp.product_id = :product_id FOR UPDATE;
-- Order eligibility uses shared, rather than exclusive, line/pair locks.
SELECT l.*, l.xmin FROM production_lines l
WHERE l.id = :line_id FOR SHARE;
SELECT lp.* FROM production_line_products lp
WHERE lp.line_id = :line_id AND lp.product_id = :product_id FOR SHARE;

```

Hold acquired locks through commit/rollback. Product FOR SHARE blocks unit/state updates; line/pair FOR SHARE blocks non-key retirement/timing updates, unlike FK KEY SHARE alone. See [row locks](#). Missing lock rows are recorded for validation after parent/version precedence, not invented. No parallel EF operations on the shared context/connection.

3.3 Aggregate persistence invariant

Before persistence, build a validated mutation plan without partial SaveChanges. Apply only allowed fields/actions; no pair delete/restore, key/code rewrite or client assignment of confirmed revision. Force the tracked parent's updated_at_utc property modified on every aggregate save, including pair-only changes and identical clock values. Thus an actual parent UPDATE advances xmin. EF original xmin is from the locked row; compare submitted version before applying changes and retain the concurrency predicate at save. All pair changes and parent update use one transaction. Product rows are observed, not modified by this service.

4. Read-method processing

4.1 ListAsync — FN-032

Arguments: validated q/state/page plus CancellationToken. Return: `Result<LinePage>`.

Step	Processing	Calls
1	Normalize/check API bounds; compute safe offset (page-1)*50; reject unsupported fields	Application validator
2	Begin bounded read snapshot; apply active/retired/all and literal code/name ILIKE using escaped backslash, percent and underscore	Repository read transaction/query
3	Count filtered rows; order code then id; project only summary fields, up to 50	Repository count/page
4	Commit/dispose read transaction, format strings, return empty page when beyond total, classify once	Mapper/telemetry

A total of zero is valid data; a failed query never becomes zero. No new trigram index, sort identifier, cross-unit total or pageSize override.

4.2 GetAsync — FN-033/034

Arguments: id, pairsPage, CancellationToken. Return: `Result<LineDetail>` .

Step	Processing	Calls
1	Validate inputs, begin read-only snapshot; read parent including xmin	Repository detail
2	Return NOT_FOUND if parent absent; include retired parent	API result mapper
3	Count all saved pairs, page by product SKU/id; join current product unit/revision/state and compute warning	Repository pair projection
4	Detect broken references as integrity failure; map parent and pair page from same snapshot; release snapshot	Mapper/telemetry

Unit revision mismatch remains visible even after returning to the original unit. Reading/formatting does not reconfirm timing. Paging changes no persisted pair.

4.3 ProductChoicesAsync — FN-034

Arguments: q/page and optional lineId, CancellationToken. Return: `Result<ProductChoicePage>` . In one read snapshot, check line exists when provided; filter active products; use NOT EXISTS: return only products for which no saved pair exists for that line, including retired pairs. Escape SKU/name search, count/page ordered SKU then UUID. Project unit/revision together. Return 404 for unknown editing line. Unsaved draft exclusion belongs to SPD; choices do not authorize a write.

4.4 EligibleAsync — FN-036

Arguments: productId, q/page, CancellationToken. Return: `Result<EligibleLinePage>` . In one read snapshot load product observation (unknown 404). Inactive product returns empty items with that observation. Otherwise query active lines/pairs for the exact product, confirmed unit/revision equal to current product; apply literal code/name search, count/page by line code/id. Project exact coefficient and unit. No reservation; ValidateOrderLineAsync repeats eligibility under locks.

5. Write-method processing

5.1 CreateAsync — FN-033/034

Arguments: CreateLineRequest, CancellationToken. Return: `Result<LineDetail>`.

Step	Processing	Calls
1	Normalize/check code/name/hours, row count, unique product IDs, coefficient syntax, explicit confirmation and observed revision format	Application validator
2	Begin bounded READ COMMITTED transaction; lock all requested products in canonical UUID order	LockProductObservation
3	Require each product active and observed unit/revision equal to locked values; collect allowed API errors before applying any row	Timing intent validator
4	Create active line UUID v7 and UTC audits, pair identities and coefficient; derive confirmed fields from locked products, not client supplied storage values	Domain construction/repository Add
5	Save once; unique code index authoritative; commit once with phase tracking; no empty-line prohibition	Save/CommitOutcome
6	Read current committed detail page 1 in a new snapshot using remaining deadline; return 201 through API	ReadDetailProjectionAsync/mapper

Duplicate prechecks cannot authorize creation. Named code-index 23505 maps to LINE_CODE_CONFLICT; unrelated uniqueness/integrity errors are generic failures. A post-commit projection may reflect later committed changes; it is current data, not a duration/history snapshot. Projection failure does not undo the saved line. Post-commit projection shares the original remaining deadline; it does not invoke a new public GetAsync with another 15-second budget.

5.2 UpdateAsync — FN-033/034

Arguments: id, UpdateLineRequest, CancellationToken. Return: `Result<LineDetail>`.

Step	Processing	Calls
1	Validate name/hours/version and action schemas; one action per product, at most 1000; retain original array indices for safe field errors	Application validator
2	Begin write; lock touched products sorted, then parent FOR UPDATE; unknown line 404, stale parent LINE_STALE before business errors	Repository locks/version comparator
3	Lock existing affected pairs sorted; add rejects any existing pair, including retired; absent setTiming/retire pair 404, retired operation LINE_PAIR_INVALID	Pair mutation validator
4	Validate every timing intent as below; retirement requires active saved pair; untouched rows need no reconfirmation	Timing intent validator
5	Apply entire validated plan: mutable parent fields, add/setTiming/retire rows; force parent audit UPDATE; SaveChanges and commit once	Aggregate persistence/CommitOutcome

Step	Processing	Calls
6	New snapshot detail page 1 within remaining deadline; 200 with current version; no automatic merge/replay	Mapper/telemetry

Timing action	Validation / confirmed-field changes
add	Product exists/active; observed unit/revision current; confirmUnit=true; insert coefficient and locked confirmation
setTiming	Saved pair active; product exists; observed unit/revision current. If existing confirmed unit/revision stale, require confirmUnit=true; false retains already-current confirmation
setTiming with true	Assign new coefficient and locked current unit/revision only after explicit intent passes; no unit conversion
retire	Change pair is_active false and audit only; no timing/confirmation fields accepted

Inactive products may retain/edit existing timing under this contract but cannot be added anew or selected/started on an order. Retired-line corrections keep is_active false. Omitted pairs are unchanged, not removed. Stale unit observation maps LINE_UNIT_STALE; stale saved confirmation without true intent maps LINE_UNIT_CONFIRMATION_REQUIRED. No partial commit if a later action fails.

5.3 RetireAsync — FN-035

Arguments: id, parent version, CancellationToken. Return: `Result<LineSummary>`. Validate; begin write; lock parent FOR UPDATE (no product/pair write); unknown 404, stale 409, current already-retired 400 LINE_ALREADY_RETIRED. Set false/audit, force parent update, save/commit once and return current summary. Keep all pairs and orders. Cancellation of a UI dialog does not invoke this method. No restore, DELETE or implicit update of another dirty form.

6. Order eligibility integration — FN-036

ValidateOrderLineAsync

Arguments: origin order snapshot (null on create), submitted product/line presence, requested status and shared write transaction/cancellation. Return: a validated line decision or existing/new field error; it does not save/commit itself. Existing order service remains responsible for original quantity/status/date rules, order numbering, order xmin check and single transaction. No independent context, HTTP call, nested transaction or premature lock release.

Step	Processing	Calls
1	Validate lineId syntax/presence. On create omitted/null means null; on update omitted means origin value, explicit null means clear	Presence-aware request mapper
2	Existing order read/version and Domain field/status guards retain precedence; reject changed line outside origin Draft with LINE_LOCKED	Existing order service/domain plus line rule

Step	Processing	Calls
3	Determine non-null new/changed selection (line or product changed), or Draft -> InProgress; clearing a Draft line skips line/pair checks; legacy unchanged non-Draft/null requires no fresh eligibility	OrderLineDecision rule
4	For required eligibility acquire product, line and exact pair FOR SHARE in order; reuse existing product SHARE observation in that same transaction	Shared lock ports
5	Start with null -> LINE_REQUIRED; absent/inactive/mismatched selected line/pair/unit -> LINE_INELIGIBLE; product validation preserves existing codes	Eligibility rule/API mapping
6	Apply validated lineid with existing order changes; SaveChanges retains order xmin condition; commit once while eligibility locks held	Existing order transaction

For a Draft product change, a retained non-null pair must match the new product; server does not invent a replacement. Keep existing unchanged Draft/history retirement exceptions, but Draft start always rechecks full active eligibility. Outside Draft preserve legacy null during otherwise allowed edits. Unchanged non-Draft line values may be explicitly resubmitted; different values are locked.

Existing order reads currently use optimistic xmin rather than an exclusive row lock. Keep that behavior: if another edit/status transition wins after the origin read, the final xmin predicate fails, rolling back every proposed change. Do not commit based only on an old status read. New integration acquires product/line/pair locks before the order UPDATE's implicit lock; no order FOR UPDATE followed by product acquisition. Unit edits hold product FOR UPDATE and retain the WI-006 reference restriction; the planned trigger advances revision on actual unit changes, including changed-back units. This service never writes unit_revision.

Order detail/list projection adds current line or real legacy null. Non-null missing line/pair is an integrity failure, not an invented unassigned value. Project efficiently with the approved nullable reference; no per-row lookup loop. Full route/client compatibility states remain in the later new order-impact DD.

7. Commit outcomes, error handling and disposal

A write tracks before_commit, commit_in_flight, committed or unknown. Set commit_in_flight immediately before calling CommitAsync. Mark committed only when commit acknowledgement returns. A failing/readback-lost response cannot be treated as safe write retry merely because the client received no success.

Failure point / condition	Outcome / required handling
Validation or missing/stale target before persistence	Roll back/dispose transaction; return exact API error; no persisted action
SaveChanges optimistic concurrency / named code uniqueness	Roll back; LINE_STALE / LINE_CODE_CONFLICT; no replay
55P03 lock wait, 40P01 deadlock or 57014 timeout before commit, rollback confirmed	LINE_BUSY 503 with Retry-After: 1; explicit user retry allowed
Request disconnected before commit	Cancel operation, bounded rollback/disposal; record cancelled, no response guarantee

Failure point / condition	Outcome / required handling
Rollback cannot be confirmed / transport fails during commit	Unknown/error classification; generic failure when response possible; never LINE_BUSY as a safely retryable write
Commit succeeded but projection/response fails	Data remains committed; client outcome ambiguous; no compensation/delete, no replay
Unexpected FK/check/grant/integrity failure	Roll back if possible; generic UNEXPECTED; no SQL/raw exception exposure

Do not reuse a request-scoped context with failed tracked changes. Dispose transaction/context; cleanup cannot reuse the cancelled request token. If cleanup or connection state is uncertain, provider-supported connection disposal must prevent a pending/broken transaction returning to usable pool state. No pool-wide reset or schema repair is triggered by a user request. No retry-on-savepoint loop, background replay or implicit idempotency guarantee. SQLSTATE classification requires actual provider evidence; a generic timeout message is not proof.

8. Instrumentation sites and secure diagnostics

Use the API-approved ActivitySource/Meter ProductionManagementAI.ProductionLines, registered in the existing pipeline. Span names are ProductionLine.List, .Get, .Create, .Update, .Retire, .ProductChoices, .Eligible. Start at method entry after transport authorization. FN-036 adds a ProductionLine.ValidateOrder child span inside the existing order trace; it does not create a second HTTP request count.

Site	Measurement / completion rule
Feature boundary rejected authentication/authorization/body binding	Record one approved operation/outcome sample; missing session maps forbidden outcome with HTTP status distinction
Valid use-case invocation	Start named span and monotonic timer; all returns/exceptions reach one completion guard
Locks/intent validation	Bounded span events for lock phase and validation outcome; no IDs/values or SQL parameters
Commit result / projection result	Classify commit ambiguity before final completion; success only for delivered successful operation result
Finally	Increment requests counter and duration histogram once with API-approved operation/outcome; include cancellation/error paths
Failure diagnostic	Structured operation, outcome and traceId only; do not serialize exception objects containing SQL/values

Use only API operation/outcome label sets. Bounded HTTP route templates/status may remain standard instrumentation; redact query/raw URL attributes containing search values. Never attach coefficient/name/code/cookie/version/body or IDs as metric labels; do not enable EF sensitive-data logging or Npgsql parameter capture. Conflict/business rejection is an expected outcome; unexpected failure marks span error without exception text. OTLP export stays under existing configuration. No trace export or instrumentation behavior has been executed in this phase.

9. Verification viewpoints and review handoff

Requirement / risk	Later verification
REQ-064/065	Snapshot count/page consistency, deterministic order, query bounds, duplicate code race and stale parent precedence
REQ-066	Exact decimals, failed later action rolls back all rows, same-clock pair edit forces parent UPDATE, ABA and observed-unit race
REQ-067	Retire/selection serial order, saved rows preserved, unknown commit no replay or compensating delete
REQ-068	Optional Draft, eligible start, omitted vs null, concurrent status change/order xmin, legacy null/history and product-change compatibility
REQ-069	Admin/Operator API gates, restricted column grants, deadline cleanup, no double metrics, bounded labels/no secrets

No business decision remains open. Lock/cleanup/method organization are technical review proposals consistent with approved inputs. Source/document checks are not runtime SQL, concurrency, migration, instrumentation or application-test evidence. The SPD and order-impact DD still need sequential review; full family/security and implementation-ready gates remain pending. Submit this file with current EN/JA PDFs and wait for explicit review before writing DD-SPD. Application code requires separately approved plan revision 2; no approved design was revised.